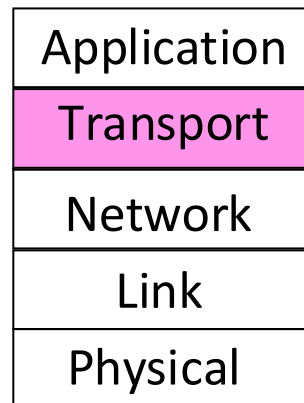


Capítulo 3

Capa de Transporte Transferencia de datos confiable



Metas

- Ejercitaremos los siguientes asuntos:
 1. Entrega de datos confiable
 2. Protocolo de parada y espera
 3. Protocolos de tubería
 4. Control de flujo en la capa de transporte
 5. Control de flujo en TCP

Entrega de datos confiable

Situación: los mecanismos básicos (ACK, temporizadores, retransmisiones) no son suficientes por sí solos. Quedan preguntas abiertas:

- ¿Qué se confirma exactamente en un ACK? ¿**Un paquete o varios?**
- ¿**Cuántos paquetes se pueden enviar antes de recibir un ACK?**
- ¿Qué hace el receptor cuando un paquete se pierde o llega dañado?

Respuesta: se necesita un **protocolo de entrega confiable** que defina reglas precisas para el emisor y receptor.

¿Por qué hay varios protocolos? Porque el protocolo óptimo depende de las **características de la red**:

- **Latencia** — cuánto tarda un paquete en llegar
- **Tasa de errores** — qué tan frecuente es la pérdida/corrupción
- **Capacidad** — tasa de datos que puede manejar la red

Marco de los protocolos de entrega confiable

La **capa de transporte** debe incluir al menos un protocolo de **entrega confiable**. Estudiaremos varios, de menor a mayor complejidad.

Supuestos sobre el canal de comunicación:

- Puede **corromper** paquetes (bits alterados en tránsito)
- Puede **perder** paquetes (nunca llegan al destino)
- La transferencia es **unidireccional** (un emisor, un receptor)

Protocolos que veremos (de simple a complejo):

- **Parada y Espera** — el más simple, un paquete a la vez
- **Tubería (Pipelining)** — múltiples paquetes en vuelo (Retrosceso-N, Repetición Selectiva)

Nota: estos protocolos aplican tanto a la capa de transporte como a la capa de enlace — la entrega confiable es un problema común a ambas.

También la **capa de enlace** incluye protocolos de entrega confiable enfocados en un solo salto, mientras que los de la capa de transporte lo hacen extremo a extremo.
(Los vemos después!)

Herramientas básicas de la entrega confiable

La Capa de Transporte dispone de tres mecanismos fundamentales:

1. ACK (confirmación de recepción)

- El receptor envía un paquete ACK para confirmar que recibió los datos correctamente.
- Si el emisor recibe el ACK → sabe que el paquete llegó bien.

2. Temporizadores (timers)

- ¿Cómo saber si un paquete se perdió? Si pasa cierto tiempo sin recibir ACK.
- El temporizador mide ese tiempo de espera máximo (timeout).

3. Retransmisiones

- Si el temporizador expira sin ACK → el emisor retransmite el paquete.

Problema: paquetes duplicados

Situación: se **perdió un ACK**, el emisor retransmite, pero el paquete original sí llegó al receptor.

Problema: el receptor recibe el **mismo paquete dos veces** y lo entrega a la aplicación duplicado.

- **Esto es inaceptable** — la aplicación recibiría datos corruptos (ej: transferencia bancaria duplicada).

Solución: números de secuencia

- Cada paquete lleva un **número de secuencia** único.
- Cuando llega un paquete, el receptor revisa si ya recibió ese número.
- Si es **duplicado** → **lo descarta**. Si es **nuevo** → **lo entrega** a la aplicación.

Metas

- Ejercitaremos los siguientes asuntos:
 1. Entrega de datos confiable
 2. **Protocolo de parada y espera**
 3. Protocolos de tubería
 4. Control de flujo en la capa de transporte
 5. Control de flujo en TCP

¿Cuándo usar Parada y Espera?

Situación: la latencia de la red es **baja**.

- El RTT (**Round Trip Time** o tiempo de ida y vuelta) es bajo comparado con el tiempo de transmitir un paquete.
- **Consecuencia:** el ACK llega antes de que se termine de preparar el siguiente paquete.
- → Solo se puede tener **un paquete "en vuelo"** a la vez.

Protocolo óptimo: Parada y Espera (Stop-and-Wait)

- Envía un paquete y se detiene hasta recibir confirmación.
- Si llega ACK → envía el siguiente paquete.
- Si expira el temporizador → retransmite el mismo paquete.
- **Es el protocolo más simple — ideal cuando RTT es bajo.**

Lo usa WiFi (pero en capa de enlace!)

Protocolo de Parada y Espera

Supuestos del protocolo:

- El canal puede perder paquetes de datos y ACKs

Mecanismos usados:

- **N° de secuencia**: 1 bit basta (alterna 0/1) para detectar duplicados
- **ACKs**: incluyen el N° de secuencia del paquete confirmado
- **Temporizadores**: detectan pérdidas y disparan retransmisiones

Comportamiento del emisor:

1. Envía paquete P y **para** (no envía más hasta recibir respuesta)
2. **Espera** un tiempo razonable (timeout)
3. Si llega ACK → envía siguiente paquete (volver a 1)
4. Si expira timeout → retransmite P (volver a 2)

¿Y si el ACK llega tarde?

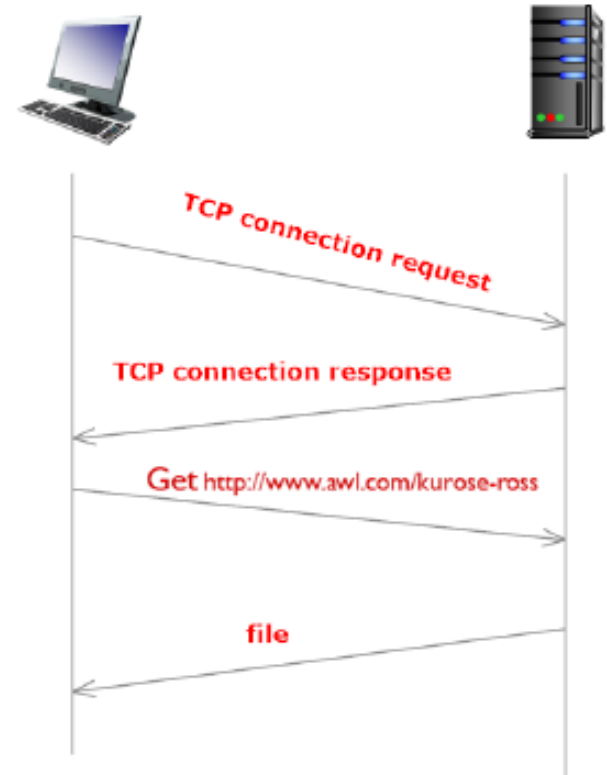
- Se retransmite un duplicado → el receptor lo detecta por el N° de secuencia y lo descarta.

Protocolos para entrega de datos confiable

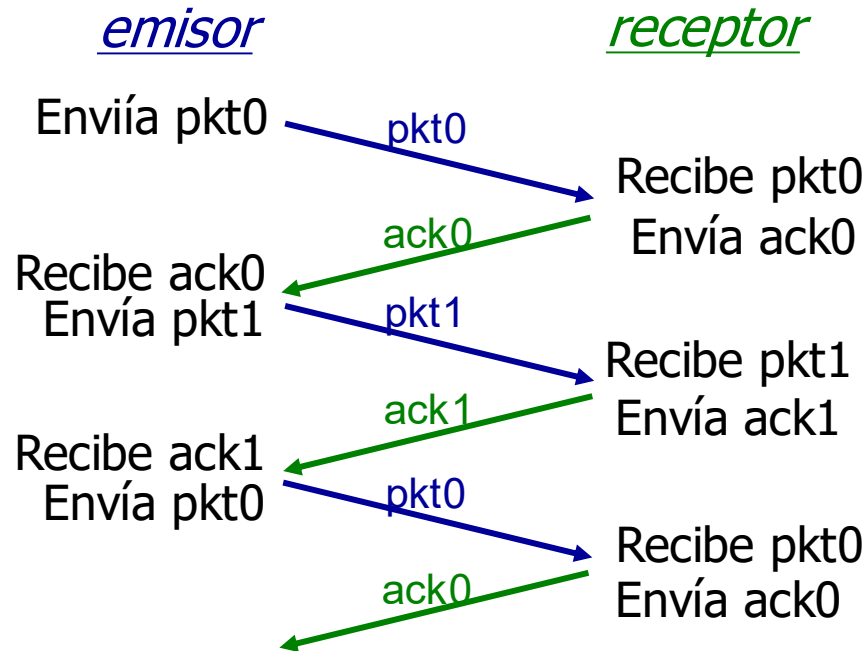
Diagramas de secuencia de mensajes

- Hay máquinas con diferentes roles
- Mensaje entre par de máquinas
- Se describe de arriba hacia abajo secuencia de esos mensajes

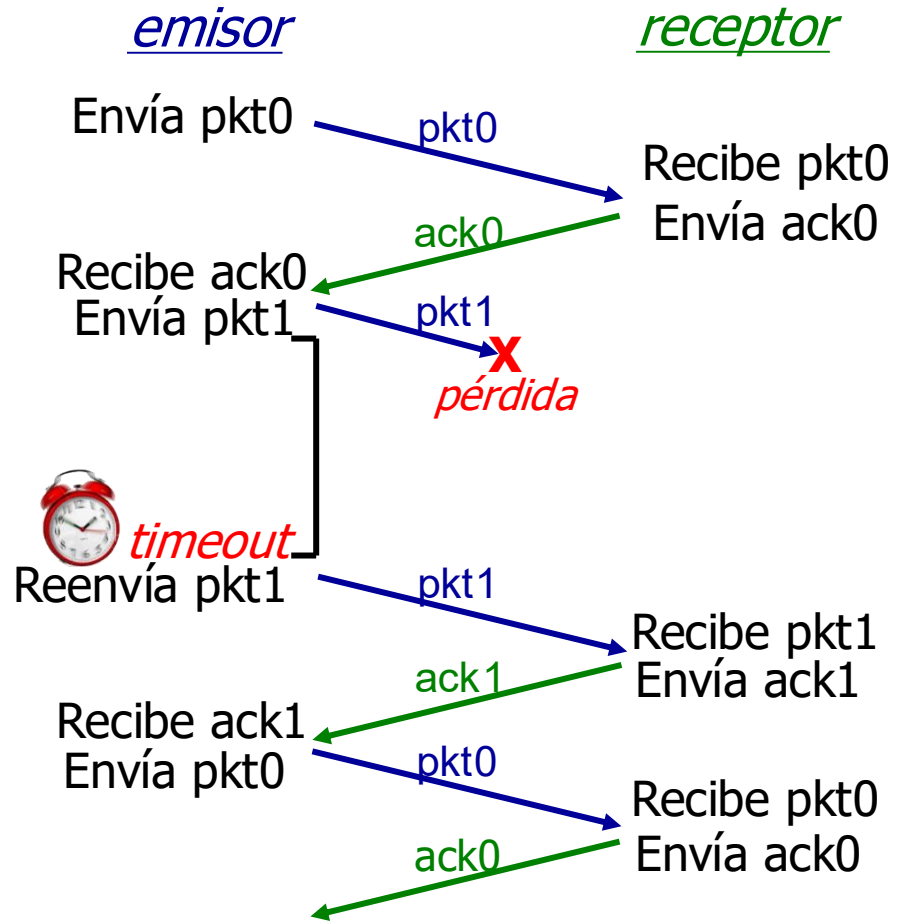
Ejemplo: figura a la derecha



Parada y Espera en Acción



(a) Sin pérdida

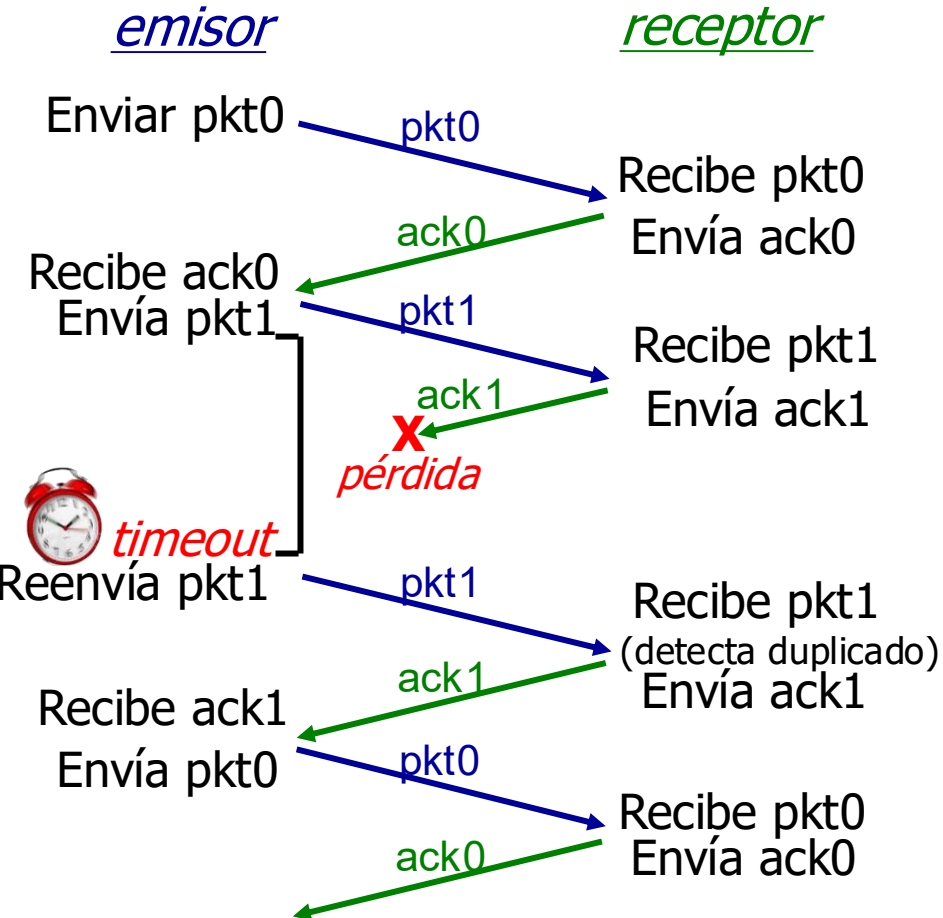


(b) Pérdida de paquete

- ¿Qué pasa si se pierde el pkt1?

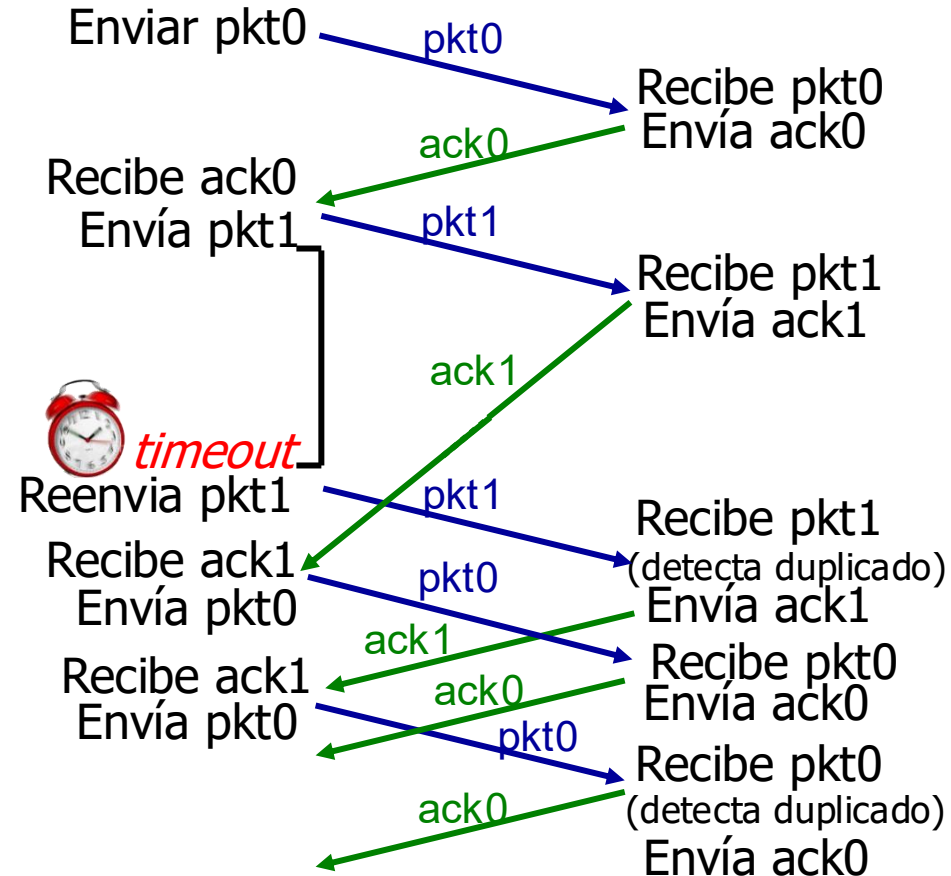
Parada y Espera en Acción

- ¿Qué pasa si se pierde el ack1?



(c) Pérdida de ACK

- ¿Qué pasa si el ack1 se demora?
emisor receptor



(d) Timeout prematuro/ ACK demorado

Evaluando Parada y Espera: simplificaciones

¿Cómo evaluar si Parada y Espera es eficiente?

Haremos un **análisis de mejor caso** con **simplificaciones**. Si aun en el mejor caso el rendimiento es bajo, entonces el protocolo no es adecuado.

Simplificación del canal:

- Un solo enlace directo entre emisor y receptor (ej: un cable).
- Sin routers intermedios — solo retardo de propagación.

Supuestos de mejor caso:

- No se pierden paquetes
- No hay demoras adicionales
- No se corrompen datos en tránsito

Variables del análisis de Parada y Espera

L = longitud del segmento (bits)

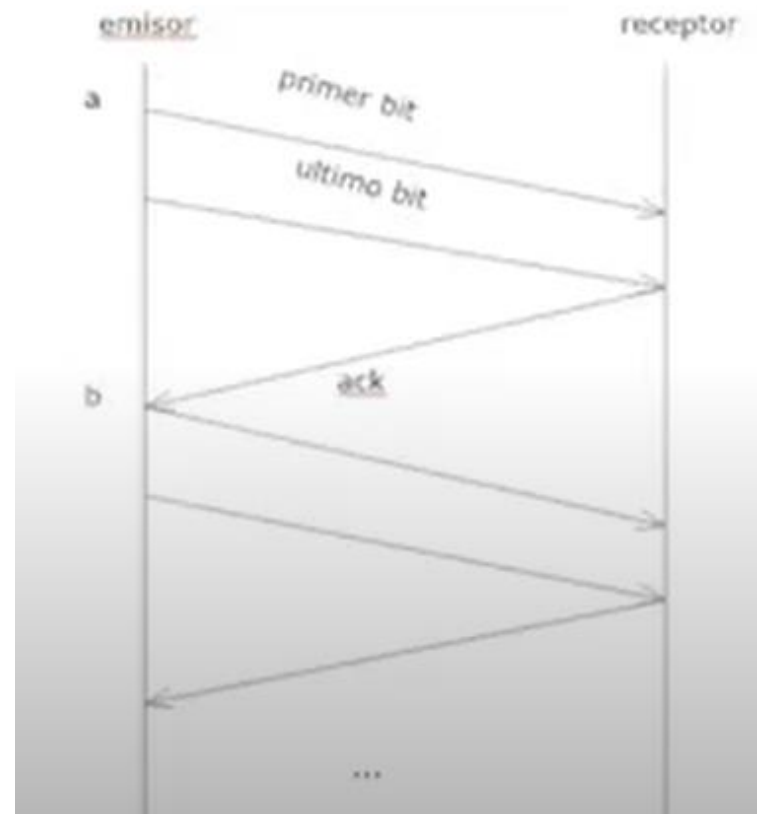
T = tasa de transmisión (bits/seg)

D = demora de propagación

RTT = ida y vuelta = $2D$

Preguntas clave:

- ¿Cuánto tarda enviar un paquete?
 - → **Tiempo de transmisión**
 $= L / T$
- ¿Cuánto del RTT se calcula con D?
 - → **RTT**
 $= 2 \times D$ (ida + vuelta)

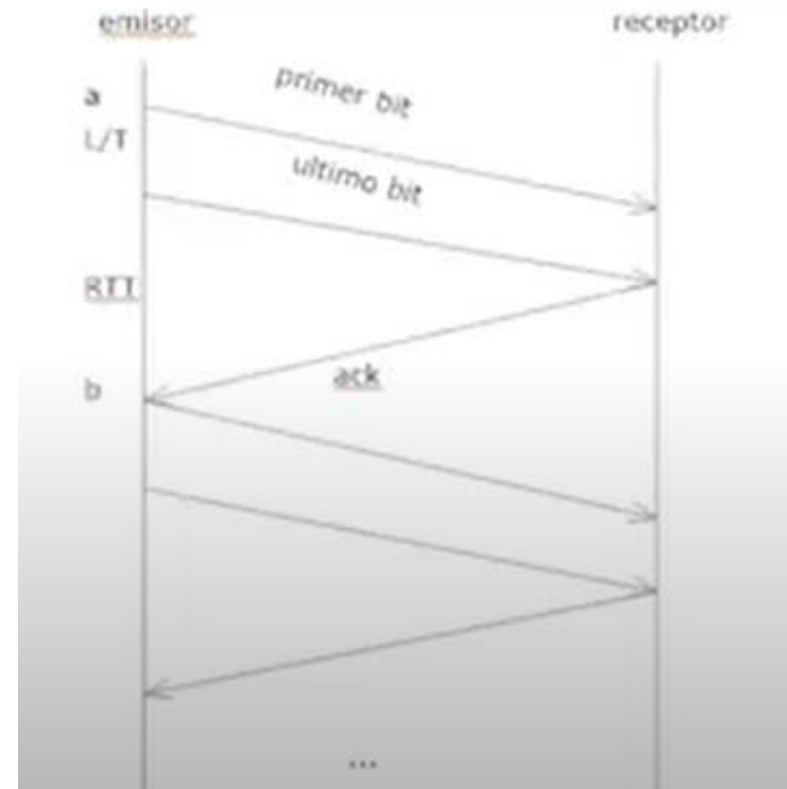


Protocolo de parada y espera

- Tiempo de transmisión del segmento: L/R
- $RTT = 2 * D$
- Utilización del canal U_{sender}

$$U_{\text{sender}} = \frac{L/R}{RTT + L/R}$$

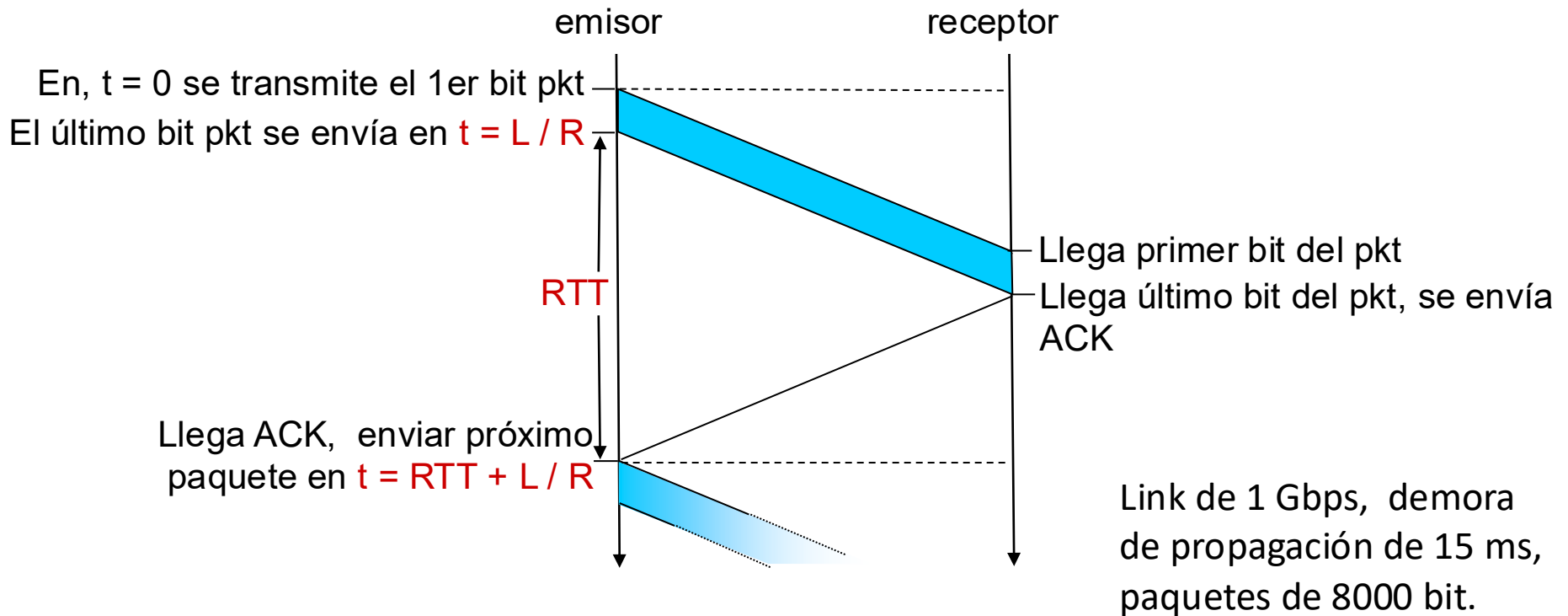
Entre 0 y 1
(se puede expresar en %)



Desempeño de Parada y Espera

- **Ejemplo concreto:** enlace de **1 Gbps**, propagación **15 ms**, paquetes de **8000 bits**
 - $D_{envío}$ es la demora en enviar un paquete.
 - U_{sender} : **utilización** – es la fracción del tiempo en que el emisor está ocupado enviando.
 - RTT es tiempo de ida y vuelta de un bit: RTT = 30 msec.
- El protocolo de red limita el uso de recursos físicos.

Desempeño de Parada y Espera



$$D_{\text{envío}} = \frac{L}{R} = \frac{8000 \text{ bits}}{10^9 \text{ bits/sec}} = 8 \text{ microsecs}$$

Suposición: RTT fijo

$$U_{\text{sender}} = \frac{L / R}{RTT + L / R} = \frac{.008}{30.008} = 0.00027$$

Metas

- Ejercitaremos los siguientes asuntos:
 1. Entrega de datos confiable
 2. Protocolo de parada y espera
 3. **Protocolos de tubería**
 4. Control de flujo en la capa de transporte
 5. Control de flujo en TCP

De Parada y Espera a Tubería (Pipelining)

- **Situación:** la latencia de la red es **alta**.
 - El **RTT** es mucho mayor que el tiempo de transmitir un paquete.
 - → Se pueden enviar múltiples paquetes antes de recibir el primer ACK.
- **Problema con Parada y Espera:**
 - Si usamos Parada y Espera con alta latencia, el emisor pasa la mayor parte del tiempo ocioso esperando el ACK → **utilización muy baja del enlace**.
- **Solución: Protocolo de Tubería (*Pipelining*)**
 - Permite enviar múltiples paquetes sin esperar confirmación de cada uno.
 - Mantiene el "**tubo**" **lleno** → aprovecha mejor el ancho de banda disponible.
 - Necesita: **números de secuencia más grandes, buffers, y reglas de retransmisión más complejas**.

Protocolos de tubería: dos enfoques clave

Cuando se usa pipelining, el emisor tiene múltiples paquetes en tránsito. ¿Qué pasa si uno se pierde? Existen dos estrategias:

Retroceso-N (Go-Back-N)

- ▶ Ventana de N paquetes sin confirmar
- ▶ Si se pierde un paquete: retransmite ese y todos los siguientes
- ▶ Receptor: solo acepta paquetes en orden (ACK acumulativo)
- ▶ Más simple, pero puede desperdiciar ancho de banda

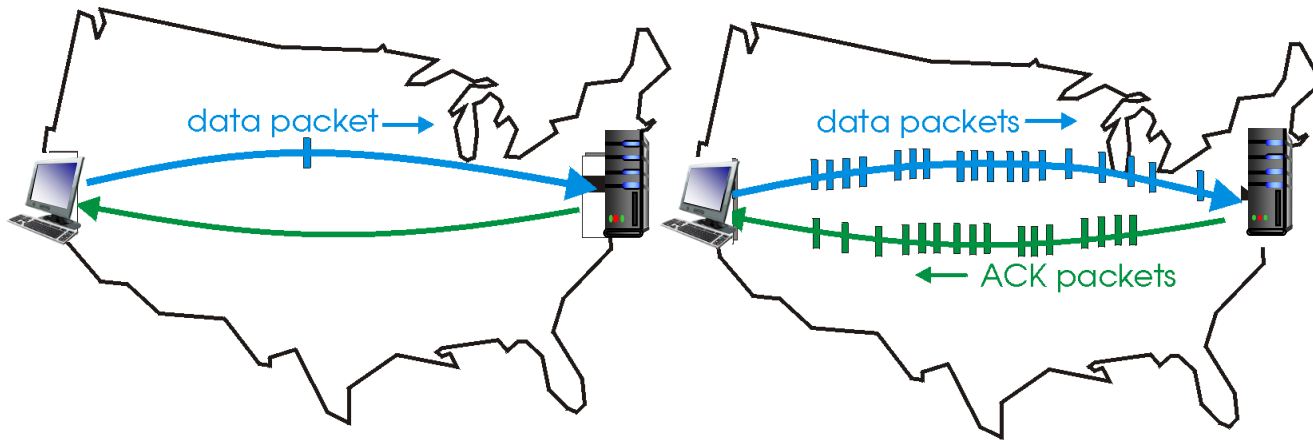
Repetición Selectiva

- ▶ Ventana de N paquetes sin confirmar
- ▶ Si se pierde un paquete: retransmite solo ese paquete
- ▶ Receptor: acepta paquetes fuera de orden (buffer + ACK individual)
- ▶ Más eficiente, pero más complejo (buffer en receptor)

Protocolos de tubería

Tubería: el emisor puede enviar múltiples paquetes al vuelo a ser confirmados

- El rango de números de secuencia debe incrementarse usando palabras de más de un bit.
- Hay que usar búferes en el emisor.

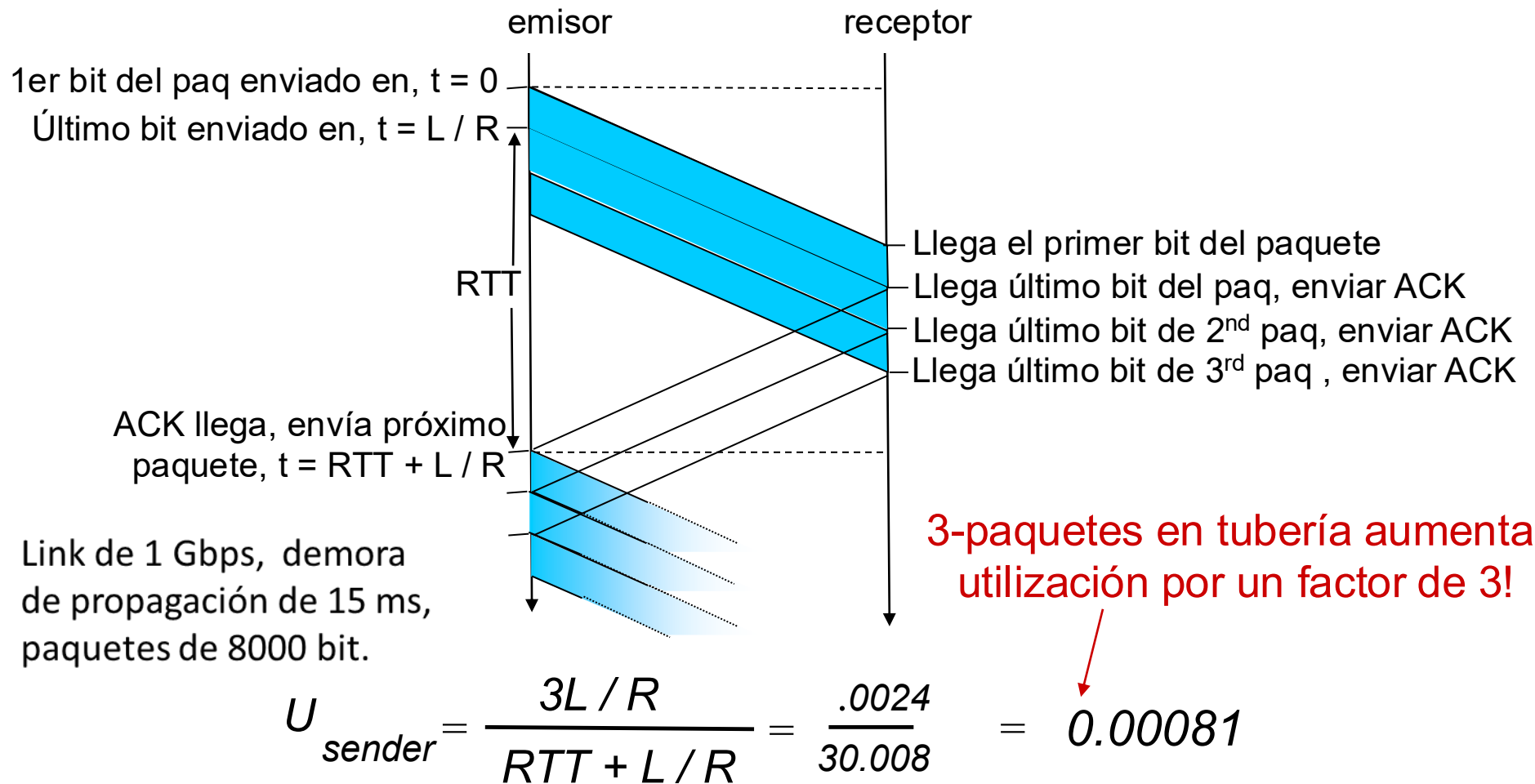


(a) a stop-and-wait protocol in operation

(b) a pipelined protocol in operation

- Hay dos formas genéricas de protocolos de tubería:
 - **retroceso N y repetición selectiva**

Tubería: utilización incrementada



Suposición: el RTT es fijo y no varía (p.ej: dos hosts unidos por cable).

Uso de búferes en el emisor

- La **ET emisora** **debe** manejar **búferes para los mensajes de salida**.
- **Esto es necesario porque:**
 - puede hacer falta retransmitirlos
- **¿Cómo se usan búferes en el emisor?**
 - El emisor almacena en búfer todas los segmentos hasta que se confirma su recepción.

Dos protocolos de tubería: ¿cuál elegir?

Ambos protocolos usan tubería (pipelining), pero difieren en cómo manejan los paquetes perdidos:

1. Protocolo Retroceso-N (Go-Back-N)

- **Receptor simple:** descarta paquetes fuera de orden
- Ideal cuando la tasa de errores es baja

2. Protocolo de Repetición Selectiva

- **Receptor más complejo:** guarda paquetes fuera de orden en buffer
- Mejor cuando la tasa de errores es significativa

La elección depende de las condiciones de la red.

- Comenzaremos estudiando Retroceso-N por ser más simple.

Retroceso-N: ¿cuándo es la mejor opción?

Situación ideal para Retroceso-N:

- La latencia es grande (RTT alto → se justifica el pipelining)
- La proporción de errores o pérdida de paquetes es muy baja
- Rara vez se demoran los paquetes

Razonamiento:

- Si los errores son raros, el receptor puede ser **más simple y eficiente**.
- No necesita buffer para reordenar paquetes → solo descarta los que llegan fuera de orden.
- El costo de retransmitir todo es bajo porque rara vez ocurre.

Nota: si los errores son frecuentes, la Repetición Selectiva será más eficiente (se verá después).

Retroceso-N: ¿qué pasa si se pierde un paquete?

Escenario: el emisor envía paquetes **1, 2, 3, 4, 5...** pero el paquete **3** se pierde.

Restricción clave: la CT receptora **debe entregar datos en orden** a la capa de aplicación.

- Los paquetes 4 y 5 llegaron bien, pero **no pueden entregarse** porque falta el **3**.

Pregunta central: ¿qué hacer con los paquetes correctos que llegaron después del perdido?

- Opción A (**Retroceso-N**): descartarlos → **más simple**
- Opción B (**Repetición Selectiva**): guardarlos en buffer → **más eficiente**

Retroceso-N: estrategia del receptor

Solución de Retroceso-N ante una pérdida:

- El receptor **descarta todos los paquetes** posteriores al perdido.
- No envía ACK para los paquetes descartados.

¿Qué pasa después?

- El emisor no recibe ACK → expira el temporizador.
- El emisor "retrocede" y retransmite **todos** los paquetes desde el perdido en adelante.

¿Por qué "Retroceso-N"?

- Porque el emisor "retrocede" N posiciones en la ventana y reenvía desde ahí.

Retroceso-N: comportamiento del receptor

El receptor usa **ACK acumulativo**: confirma hasta el mayor N° de secuencia recibido en orden.

Al recibir paquete n:

Caso 1: n está en orden y correcto

- → Envía ACK(n) y entrega los datos del paquete n a la capa superior

Caso 2: n está fuera de orden, dañado o duplicado

- → Descarta paquete n y reenvía ACK del último paquete recibido en orden

Clave: el receptor no necesita buffer — es muy simple.

Retroceso-N: comportamiento del emisor

El emisor mantiene un **único temporizador** para el paquete más antiguo no confirmado.

Si expira el temporizador:

- → Retransmite **todos** los paquetes no confirmados (desde el más antiguo)

Si llega un ACK nuevo:

- Si quedan paquetes sin confirmar → reinicia el temporizador
- Si todos están confirmados → detiene el temporizador

Retroceso-N: búferes y ventana del emisor

Supuestos del emisor:

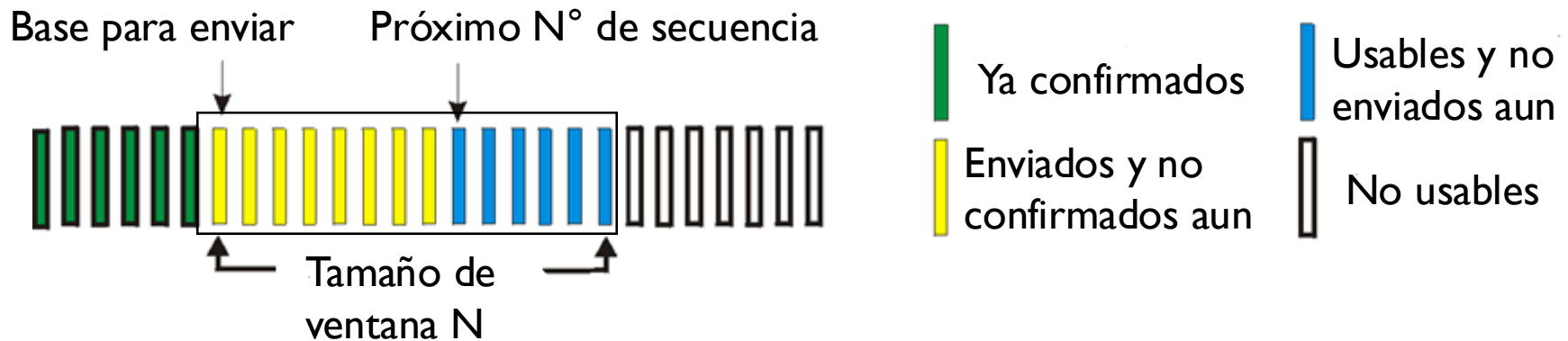
- Todos los búferes son del **mismo tamaño** (un paquete por búfer).
- La cantidad de búferes es **fija** — calculada para llenar el "tubo" durante un RTT.

Concepto clave: ventana del emisor

- Es el conjunto de N° de secuencia asignados a esos búferes.
- Define cuántos paquetes puede tener "en vuelo" simultáneamente.
- Se "desliza" hacia adelante cuando llegan ACKs confirmando paquetes.

Retroceso-N: en el emisor

- La “ventana” permite hasta N paquetes consecutivos sin confirmar
- ventana emisora** = tramas enviadas sin ack positivo o tramas listas para ser enviadas.



- $timeout(n)$: retransmite paquete n y todos los paquetes de mayor N° de secuencia en la ventana.

Retroceso-N: ACK acumulativo y *expectedSeqnum*

¿Qué N° de secuencia lleva el ACK?

- El ACK lleva el N° de secuencia más alto recibido **en orden consecutivo**.
- Esto se llama **ACK acumulativo**: confirma todo hasta ese punto.

¿Qué pasa si se pierde un paquete intermedio?

- Los paquetes siguientes generan **ACKs duplicados** (repiten el último ACK en orden).
- Ej: si llegaron 1,2,4,5 → ACK dice "2" (el 3 falta).

Variable clave del receptor:

- ***expectedSeqnum*** = N° de secuencia más chico que aún no llegó.

Retroceso-N en acción

Ventana window (N=4)

0 1 2 3 4 5 6 7 8
0 1 2 3 4 5 6 7 8
0 1 2 3 4 5 6 7 8
0 1 2 3 4 5 6 7 8

0 1 2 3 4 5 6 7 8
0 1 2 3 4 5 6 7 8

0 1 2 3 4 5 6 7 8
0 1 2 3 4 5 6 7 8
0 1 2 3 4 5 6 7 8
0 1 2 3 4 5 6 7 8

emisor

Envía pkt0
Envía pkt1
Envía pkt2
Envía pkt3
(espera)

rcv ack0, envía pkt4
rcv ack1, envía pkt5

ignore duplicate ACK



pkt 2 expira

Envía pkt2
Envía pkt3
Envía pkt4
Envía pkt5

receptor

Loss = pérdida

Recibe pkt0, envía ack0
Recibe pkt1, envía ack1

Recibe pkt3, descarta,
(re)envía ack1

Recibe pkt4, descarta,
(re)envía ack1

Recibe pkt5, descarta,
(re)envía ack1

rcv pkt2, entrega, send ack2
rcv pkt3, entrega, send ack3
rcv pkt4, entrega, send ack4
rcv pkt5, entrega, send ack5

X loss

Retroceso-N: tamaño máximo de la ventana emisora

Pregunta: si el espacio de N° de secuencia tiene $\text{MAX_SEQ} + 1$ valores (0 a MAX_SEQ), ¿puede la ventana emisora ser de tamaño $\text{MAX_SEQ} + 1$?

Respuesta: NO. La justificación se verá en las próximas 2 slides.

Conclusión: el tamaño máximo de la ventana emisora es **MAX_SEQ** (no $\text{MAX_SEQ} + 1$). Si fuera igual al espacio de secuencia completo, el receptor no podría distinguir paquetes nuevos de retransmisiones.

Retroceso-N: ¿por qué limitar la ventana?

Escenario problemático (MAX_SEQ = 7):

1. El emisor envía paquetes 0, 1, 2, ..., 7 (8 paquetes = toda la ventana).
2. Llega ACK acumulativo para paquete 7 → todos confirmados.
3. Emisor envía otros 8 paquetes con N° de secuencia 0, 1, 2, ..., 7 (se reciclan).
4. Llega otro ACK para paquete 7.

¡Ambigüedad! El emisor no puede distinguir:

- ¿Es un reenvío del ACK del paso 2? (los paquetes del paso 3 se perdieron)
- ¿O es un ACK nuevo confirmando los paquetes del paso 3?

Retroceso-N: restricción de tamaño de ventana

Del escenario anterior:

- En ambos casos (éxito o pérdida del 2º bloque), el receptor envía ACK 7.
- El emisor no puede distinguir un caso del otro.

Conclusión:

- El tamaño de la ventana emisora **no puede superar MAX_SEQ** cuando hay MAX_SEQ + 1 números de secuencia.
- Esto garantiza que nunca se reciclen N° de secuencia mientras haya paquetes sin confirmar → elimina la ambigüedad.

Limitación de Retroceso-N → necesidad de algo mejor

Problema principal de Retroceso-N:

- El **uso ineficiente del canal** cuando hay segmentos perdidos o demorados.
- Un solo paquete perdido obliga a retransmitir todos los siguientes, aunque hayan llegado bien.
- En redes con alta tasa de pérdida, esto desperdicia mucho ancho de banda.

→ Esto motiva el protocolo de Repetición Selectiva

- Retransmite solo el paquete perdido, no todos los siguientes.

Segundo protocolo de tubería: Repetición Selectiva

Protocolos de tubería estudiados:

- Protocolo Retroceso-N (Go-Back-N) ✓
- **Protocolo de Repetición Selectiva** ← siguiente

¿Cuándo usar Repetición Selectiva?

Situación:

- La latencia es alta (como en GBN).
- Además: la tasa de errores/pérdida de paquetes es significativa.
- Los paquetes pueden demorarse y llegar fuera de orden.

Problema con GBN en este escenario:

- **Con muchos errores, GBN retransmite demasiados paquetes correctos** → desperdicio.

Enfoque de Repetición Selectiva:

- Receptor más complejo: acepta paquetes fuera de orden, usa buffer, envía ACK individuales.
- **Tradeoff: código más complejo, pero mucho más eficiente con alta tasa de errores.**

Rep. Selectiva: el mismo problema, otra solución

Escenario: el paquete T se pierde a mitad de una serie larga. Los paquetes siguientes llegan bien.

Restricción: la CT debe entregar paquetes **en orden** a la aplicación → no puede entregar los que llegaron después de T .

Pregunta: ¿qué hacer con los paquetes correctos que llegaron después del perdido?

- En GBN: se descartan todos → simple pero ineficiente
- **En SR: se guardan en buffer** → solo se retransmite el paquete perdido

Repetición Selectiva: buffer en el receptor

Estrategia del receptor (a diferencia de GBN):

- Los paquetes correctos que llegan después de un paquete dañado E **se almacenan en un buffer** (no se descartan como en GBN).

Cuando el paquete E finalmente llega correcto:

- El receptor entrega a la capa de aplicación, **en orden**, tanto E como todos los paquetes consecutivos almacenados en el buffer.

Ventaja clave: solo se retransmite el paquete perdido — no toda la ventana como en GBN.

Repetición Selectiva: retransmisiones y NAK

Mecanismo básico de retransmisión:

- El temporizador del paquete E expira → el emisor lo retransmite.
- Nota: **cada paquete tiene su propio temporizador** (a diferencia de GBN que usa uno solo). Es mas complejo!

Mejora: uso de NAK (confirmación negativa)

- El receptor detecta que falta un paquete y envía un **NAK** al emisor.
- El emisor retransmite **antes** de que expire el temporizador → **mejor rendimiento**.

Rep. Selectiva: resumen emisor y receptor

Receptor:

- Confirma **individualmente** cada paquete recibido correctamente (ACK por paquete).
- Almacena en buffer los paquetes fuera de orden hasta poder entregarlos en secuencia a la aplicación.

Emisor:

- Solo **retransmite paquetes cuyo ACK no llegó o que recibieron un NAK** (confirmación negativa).
- Mantiene un **temporizador individual** para cada paquete no confirmado (a diferencia de GBN que usa uno solo).

Repetición Selectiva: ventana del emisor

Estructura de la ventana emisora:

- Contiene N N° de secuencia consecutivos.
- Limita la cantidad de paquetes no confirmados en tránsito.

Tipos de paquetes dentro de la ventana del emisor:

- **Enviados y confirmados** — aún en ventana porque antes hay paquetes sin confirmar (no puede avanzar la ventana)
- **Enviados y NO confirmados** — en tránsito, esperando ACK
- **Listos para enviar** — en buffer, esperando turno
- *Nota: a diferencia de GBN, aquí pueden haber "huecos" de paquetes confirmados entre no confirmados.*

Repetición Selectiva: ventana corrediza del receptor

Situación: el receptor necesita almacenar paquetes que llegan fuera de orden mientras espera el paquete faltante.

Problema: ¿cómo representar qué paquetes puede almacenar el receptor?

Solución: **ventana corrediza** (sliding window)

- Un intervalo de N° de secuencia dentro del espacio total.
- Define qué paquetes el receptor está dispuesto a aceptar y guardar en buffer.
- Se "desliza" hacia adelante a medida que se entregan paquetes a la aplicación.

Rep. Selectiva: tipos de paquetes en ventana receptora

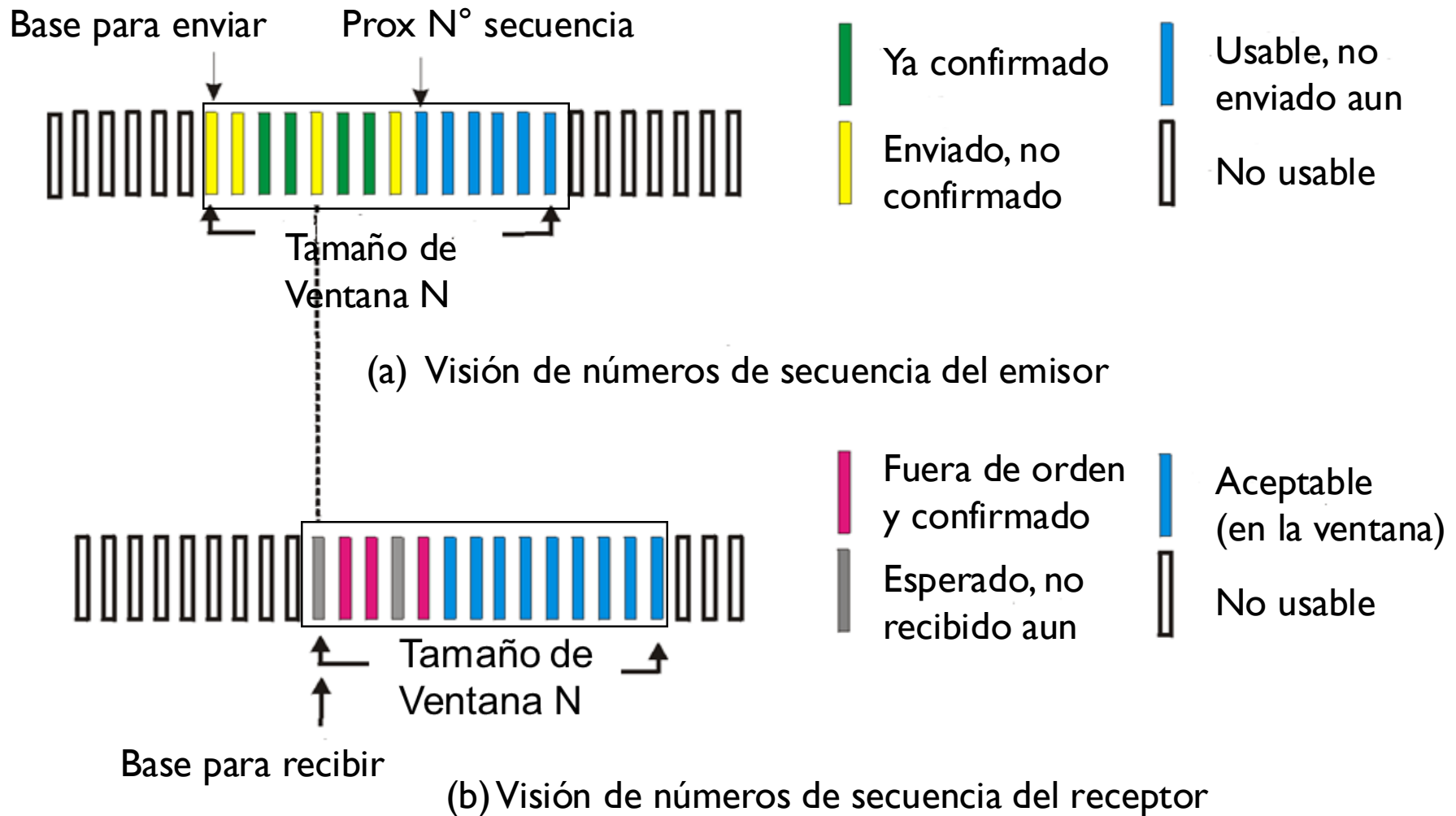
Tipos de paquetes en la ventana del receptor:

- **Esperados y no recibidos** — los que faltan (ej: el paquete perdido)
- **Recibidos fuera de orden** — almacenados en buffer, esperando al faltante
- **Aceptables pero no llegados** — dentro de la ventana, todavía en tránsito

Regla de entrega a la aplicación:

- Un paquete en buffer se entrega **solo cuando todos los que le preceden** ya fueron entregados a la capa de aplicación.

Repetición Selectiva: ventanas del emisor y del receptor



Rep. Selectiva: recepción de paquetes en el receptor

Inicialización:

- La ventana emisora comienza en tamaño 0 y crece hasta MAX_SEQ.
- El receptor tiene un buffer dedicado para cada N° de secuencia en su ventana.

¿Qué ocurre cuando llega un paquete?

- Se verifica si su N° de secuencia **cae dentro de la ventana receptora**.
- Si está dentro y no fue recibido aún → se acepta y almacena en el buffer correspondiente.
- Si está fuera de la ventana → se descarta (puede ser un duplicado).

Repetición selectiva

Emisor

Datos vienen de arriba:

- Si el próximo N° secuencia a enviar de la ventana está disponible, almacenar y enviar paquete

timeout(n):

- Reenviar paquete n , reiniciar timer

ACK(n) en $[\text{sendbase}, \text{sendbase}+N]$:

- marcar paquete n como recibido
- Si n es paquete más pequeño no confirmado, **avanzar base de ventana** al siguiente N° secuencia no confirmado.

Receptor

pkt n en $[\text{base rcv}, \text{base rcv} + N - 1]$

- Enviar ACK(n)
- Fuera de orden: almacenarlo
- En orden: entregar (también entregar paquetes en bufer en orden), avanzar ventana al siguiente paquete que no ha sido recibido aun.

pkt n en $[\text{base rcv} - N, \text{base rcv} - 1]$

- Enviar ACK(n)

Sino:

- ignorar

Súposiciones: N es tamaño de ventana del receptor. Algoritmo sin NAKs

Repetición selectiva en acción

ventana emisor (N=4)

0 1 2 3 4 5 6 7 8

0 1 2 3 4 5 6 7 8

0 1 2 3 4 5 6 7 8

0 1 2 3 4 5 6 7 8

0 1 2 3 4 5 6 7 8

0 1 2 3 4 5 6 7 8

0 1 2 3 4 5 6 7 8

0 1 2 3 4 5 6 7 8

0 1 2 3 4 5 6 7 8

0 1 2 3 4 5 6 7 8

emisor

Envía pkt0

Envía pkt1

Envía pkt2

Envía pkt3

(espera)

rcv ack0, envía pkt4

rcv ack1, envía pkt5

Registra que ack3 llegó



pkt 2 timeout

Envía pkt2

Registra que ack4 llegó

Registra que ack5 llegó

receptor

Recibe pkt0, envía ack0

Recibe pkt1, envía ack1

Recibe pkt3, almacena,
envía ack3

Recibe pkt4, almacena,
envía ack4

Recibe pkt5, almacena,
envía ack5

rcv pkt2; entrega pkt2,
pkt3, pkt4, pkt5; envía ack2

Q: ¿Qué pasa cuando llega ack2?

Dilema de repetición selectiva

Ejemplo:

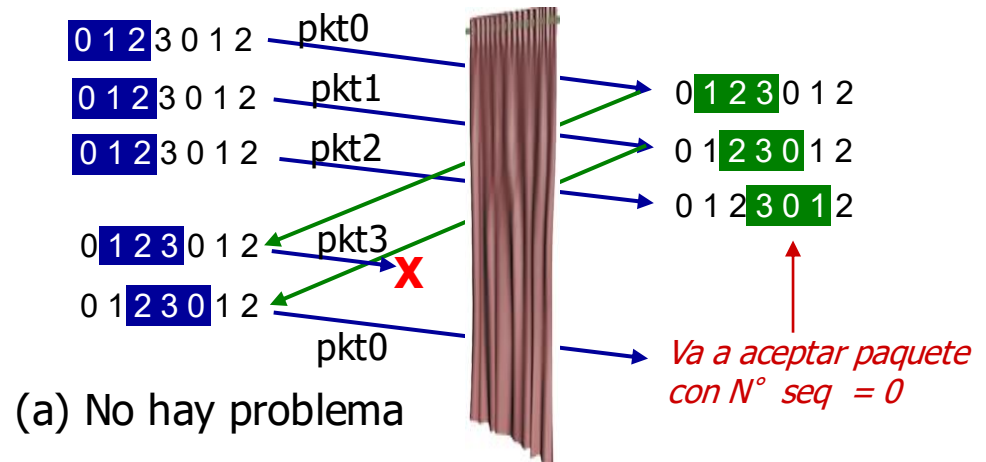
- Espacio de secuencia: 0, 1, 2, 3.
- Tamaño ventana = 3 en emisor y receptor

- El receptor no ve la diferencia en 2 escenarios.
- Datos duplicados aceptados como nuevos en (b)

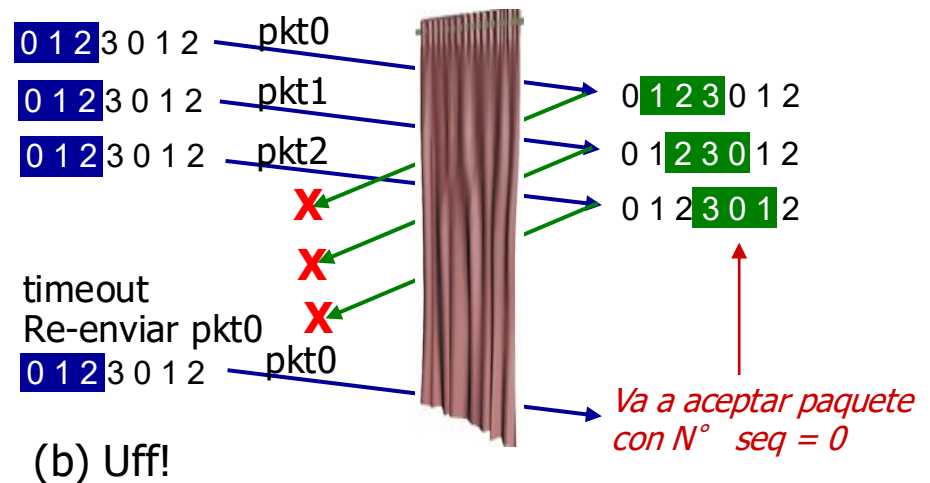
Q: ¿Qué relación entre tamaño de N° secs y tamaño de ventana en receptor debe haber para evitar el problema en (b)?

Ventana emisor
(después de recibir)

Ventana receptor
(después de recibir)



*El receptor no puede ver el lado del emisor.
¡El comportamiento del receptor es idéntico en ambos casos !
¡Algo está muy mal!*



Rep. Selectiva: tamaño máximo de ventana receptora

Regla fundamental:

- Tamaño de ventana receptora = $(MAX_SEQ + 1) / 2$

¿Por qué esta restricción?

- Con ventanas más grandes, el receptor no puede distinguir entre paquetes nuevos y retransmisiones de la ronda anterior.
- La mitad del espacio de secuencia garantiza que las ventanas de emisor y receptor nunca se solapan.
- **Con tamaños mayores de ventana: el protocolo no funciona correctamente.**

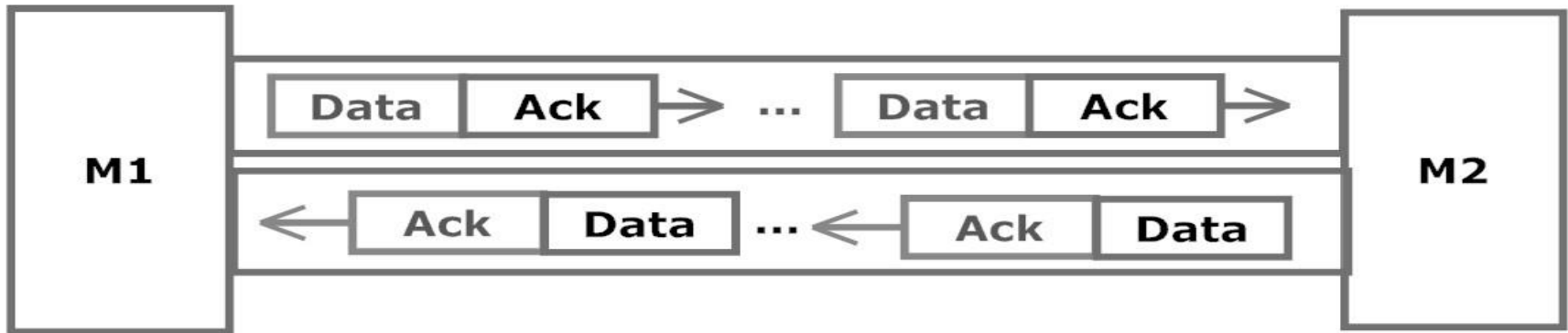
Rep. Selectiva: comunicación bidireccional

Cada paquete incluye un N° de secuencia de **k bits** en su **encabezado**.

Problema: hasta ahora asumimos transferencia unidireccional. En la práctica, ambas máquinas envían y reciben datos simultáneamente.

¿Cómo transmitir datos eficientemente en ambas direcciones?

Piggybacking: ¿cómo funciona?



- **Solución:** piggybacking (llevar a caballito)
 - Cuando llega un segmento S con datos, el receptor espera a que la aplicación le pase el siguiente paquete P para enviar.
 - El ACK de S se anexa a P usando el **campo ACK** del encabezado del segmento de salida.

Piggybacking (llevar a caballito)

Idea: en comunicación bidireccional, en lugar de enviar ACKs en paquetes separados, se puede **superponer el ACK dentro de un paquete de datos** que viaja en dirección contraria.

Cómo funciona:

- La CT espera a que haya un paquete de datos para enviar en la dirección de regreso.
- El ACK se incluye ("va a caballito") dentro de ese paquete de datos.
- **Ventaja:** reduce el tráfico en la red (menos paquetes de ACK independientes).

Temporizador auxiliar para ACKs

Problema con piggybacking: ¿qué pasa si no hay tráfico de regreso? El ACK se retrasa indefinidamente y el emisor cree que el paquete se perdió.

Solución: temporizador auxiliar (start_ack_timer)

- Al recibir un paquete en secuencia, se arranca un temporizador auxiliar.
- Si aparece tráfico de regreso antes del timeout → el ACK va a caballito (piggybacking).
- Si expira el temporizador sin tráfico → se envía un **ACK independiente**.

¿Por qué el timer auxiliar debe ser corto?

- tiempo de temporizador auxiliar $\ll$ tiempo de temporizador de retransmisiones.
 - $\ll$ significa mucho menor.
- ¿Por qué?
 - para asegurarse que la ack de un paquete correctamente recibido llegue antes que el emisor termine su temporización y retransmita el paquete.

Ejercicio 1: Preguntas Conceptuales

P1. ¿Por qué Parada y Espera necesita números de secuencia si solo envía un paquete a la vez?

- **R:** Para detectar **paquetes duplicados**. Si el ACK se pierde, el emisor retransmite. Sin N° de secuencia, el receptor no sabría si es un paquete nuevo o una copia.

P2. En Retroceso-N, ¿por qué el receptor descarta paquetes fuera de orden en lugar de almacenarlos?

- **R:** GBN usa **ACK acumulativo**: confirma hasta el último paquete recibido en orden. Guardar los desordenados obligaría a manejar un buffer complejo en el receptor, lo cual contradice la filosofía de **simplicidad del receptor** de GBN.

P3. ¿Cuál es la ventaja principal de Repetición Selectiva sobre Retroceso-N?

- **R:** SR solo retransmite el paquete perdido, no toda la ventana. En redes con **alta tasa de errores y ventanas grandes**, esto ahorra mucho ancho de banda. El costo: receptor más complejo con buffer.

Ejercicio 2: Cálculo de Utilización

Datos: Enlace de **100 Mbps**, propagación **25 ms**, paquetes de **10.000 bits**. ACK despreciable.

a) Calcule U para Parada y Espera.

- **R:** $D_{\text{envío}} = L/R = 10.000 / 100 \times 10^6 = 0,1 \text{ ms}$. $RTT = 2 \times 25 \text{ ms} = 50 \text{ ms}$.
- $U = D_{\text{envío}} / (RTT + D_{\text{envío}}) = 0,1 / (50 + 0,1) = 0,002 = 0,2\% \rightarrow$ ¡muy ineficiente!

b) ¿Cuántos paquetes en vuelo necesita un protocolo de tubería para lograr $U \geq 90\%$?

- **R:** $U = N \times D_{\text{envío}} / (RTT + D_{\text{envío}})$. Queremos $0,9 \leq N \times 0,1 / 50,1 \rightarrow N \geq 50,1 \times 0,9 / 0,1 = 450,9$
- Se necesitan al menos **N = 451 paquetes en vuelo** para alcanzar 90% de utilización.

c) ¿Cuál sería el throughput efectivo con Parada y Espera?

- **R:** $\text{Throughput} = U \times R = 0,002 \times 100 \text{ Mbps} = 200 \text{ kbps}$ (¡solo 0,2% del enlace de 100 Mbps!)

Ejercicio 3: Parada y Espera vs. Tubería

Datos: Enlace de **1 Gbps**, propagación **15 ms**, paquetes de **8.000 bits**. ACK despreciable.
 $RTT = 2 \times 15 \text{ ms} = 30 \text{ ms}$.

a) Calcule U y throughput para Parada y Espera.

- **R:** $D_{\text{envío}} = L/R = 8.000 / 10^9 = \mathbf{0,008 \text{ ms}}$
- $U = 0,008 / (30 + 0,008) = \mathbf{0,00027 = 0,027\%}$ | Throughput = $0,00027 \times 1 \text{ Gbps} = \mathbf{270 \text{ kbps}}$

b) Si se envían 3 paquetes en tubería (como en slide 22), ¿cuánto mejora?

- **R:** $U = N \times D_{\text{envío}} / (RTT + D_{\text{envío}}) = 3 \times 0,008 / 30,008 = \mathbf{0,0008 = 0,08\%}$
- Mejora: $\times 3$ respecto a P&E. Throughput = $0,0008 \times 1 \text{ Gbps} = \mathbf{800 \text{ kbps}}$. Aún muy bajo.

c) ¿Cuántos paquetes en vuelo se necesitan para $U = 100\%$?

- **R:** $1 = N \times 0,008 / 30,008 \rightarrow N = 30,008 / 0,008 = \mathbf{3.751 \text{ paquetes}}$
- **Conclusión:** Con alta velocidad y alta latencia, se necesita una ventana **enorme** para llenar el enlace. Esto exige muchos bits para N° de secuencia.

Ejercicio 4: Utilización y N° de Secuencia

Datos: Enlace de **10 Mbps**, propagación **10 ms**, paquetes de **5.000 bits**. ACK despreciable. Se usa **Retroceso-N**.

a) ¿Cuál es la utilización con Parada y Espera?

➤ R: $D_{\text{envío}} = 5.000 / 10 \times 10^6 = 0,5 \text{ ms}$. $RTT = 20 \text{ ms}$. $U = 0,5 / 20,5 = 0,0244 = 2,44\%$

b) ¿Cuántos paquetes en vuelo necesita GBN para $U \geq 80\%$?

➤ R: $0,8 \leq N \times 0,5 / 20,5 \rightarrow N \geq 20,5 \times 0,8 / 0,5 = 32,8 \rightarrow N = 33 \text{ paquetes}$

c) ¿Cuántos bits de N° de secuencia necesita GBN para esa ventana?

➤ R: GBN necesita $MAX_SEQ \geq 33$, o sea $MAX_SEQ + 1 \geq 34$ secuencias. Se requiere $2^k \geq 34 \rightarrow k = 6$ bits ($2^6 = 64$ secuencias, ventana máx = 63).

d) ¿Y si se usara Repetición Selectiva con los mismos 6 bits?

➤ R: Ventana máx SR = $(2^6)/2 = 32 \text{ paquetes}$. $U = 32 \times 0,5 / 20,5 = 0,78 = 78\%$. No alcanza el 80%: SR necesitaría 7 bits (ventana máx = 64).

Ejercicio 5: Tamaño Máximo de Ventanas

Dato: Un protocolo usa N° de secuencia de **3 bits** (secuencias 0 a 7, MAX_SEQ = 7).

a) ¿Cuál es el tamaño máximo de la ventana emisora en Retroceso-N?

➤ **R:** Ventana máx. = MAX_SEQ = **7**. No puede ser 8 (= MAX_SEQ + 1) porque el receptor no podría distinguir un paquete nuevo del paquete 0 retransmitido.

b) ¿Cuál es el tamaño máximo de la ventana receptora en Repetición Selectiva?

➤ **R:** Ventana máx. = $(\text{MAX_SEQ} + 1) / 2 = (7 + 1) / 2 = \mathbf{4}$. La mitad del espacio de secuencia garantiza que las ventanas de emisor y receptor nunca se solapen.

c) Si se usaran 4 bits para N° de secuencia (0 a 15), ¿cuántos paquetes en vuelo permitiría GBN?

➤ **R:** MAX_SEQ = $2^4 - 1 = 15$. Ventana máx. GBN = **15 paquetes**. Y para SR: ventana máx. = $(15+1)/2 = \mathbf{8 \text{ paquetes}}$.

Ejercicio 6: Escenarios y Piggybacking

P1. Escenario: Un emisor usa GBN con ventana = 4 y envía paquetes 0, 1, 2, 3. El paquete 1 se pierde. ¿Qué paquetes retransmite el emisor al expirar el temporizador?

- **R:** El receptor acepta pkt 0 (envía ACK 0), pero descarta pkts 2 y 3 (fuera de orden). El emisor retransmite **1, 2, 3** (toda la ventana desde el paquete perdido).

P2. Mismo escenario con Repetición Selectiva. ¿Qué cambia?

- **R:** El receptor almacena pkts 2 y 3 en su buffer y envía ACK individual para cada uno. Solo retransmite el **paquete 1**. Al recibirlo, el receptor entrega 1, 2, 3 en orden a la capa superior.
Ahorro: 2 retransmisiones menos.

P3. ¿Qué es piggybacking y cuándo resulta útil?

- **R:** Piggybacking = **incluir el ACK dentro de un paquete de datos** que viaja en sentido contrario. Es útil en **comunicación bidireccional** porque ahorra un paquete completo. Si no hay datos para enviar rápidamente, se usa un **temporizador auxiliar corto** para enviar el ACK solo y evitar que expire el temporizador del emisor.